

MongoDB

Day 5

Joins, Pipelines & Indexes

What you'll master:

- \$lookup — join collections
- \$unwind — flatten arrays
- Multi-field & multi-stage \$group
- Full 7-stage pipeline + \$out
- Indexes — explain, COLLSCAN/IXSCAN
- Compound & unique indexes
- Text indexes (\$text / \$search)

Today's Data

Add a field · new collection · reuse employees

students (+ subject)

15 students — Day 4 set, plus a new subject field

```
name · city · subject · marks
```

courses (new)

references students by name (the join key)

```
studentName · course · duration · fees
```

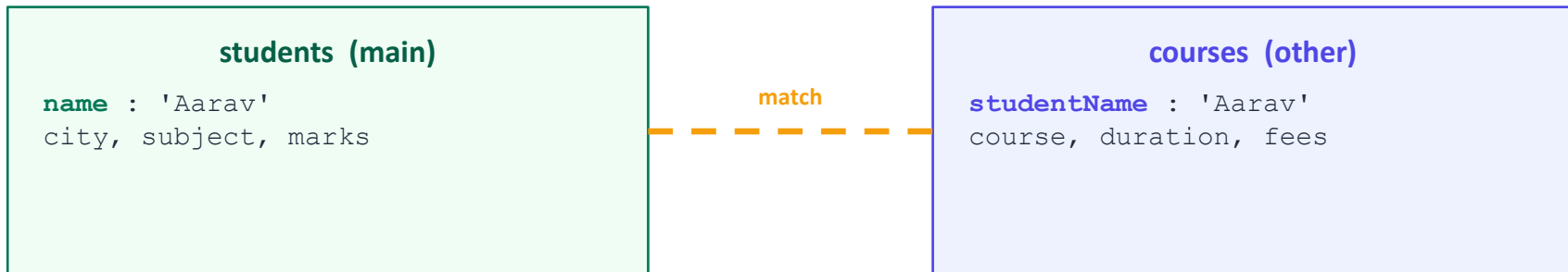
employees (reused)

from Day 1 — used for the index demos

```
name · city · role · salary · department
```

\$lookup — Joining Collections

Like a SQL JOIN: match a key across two collections



localField (primary key) = the field in your MAIN collection · `students.name`

foreignField (reference) = the matching field in the OTHER collection · `courses.studentName`

 **Join rule: link rows where `students.name == courses.studentName`**

Both collections must be in the same database (both in 'school').

\$lookup — Syntax & Result

Four fields; the matches arrive as an array

SYNTAX

```
{ $lookup: { from, localField, foreignField, as } }
```

from=other collection · localField=this key · foreignField=other key · as=new array field

```
{ from:'courses', localField:'name', foreignField:'studentName', as:'courseDetails' }
```

```
db.students.aggregate([
  { $lookup: {
    from: 'courses',
    localField: 'name',
    foreignField:
      'studentName',
    as: 'courseDetails'
  } }
])
```

Result — a new ARRAY field:

```
// Aarav (1 match):
{ name:'Aarav', marks:85,
  courseDetails: [
    { course:'Web Dev',
      fees:15000 } ] }

// Neha (no match):
{ name:'Neha',
  courseDetails: [] }
```

\$unwind — Flatten the Array

One document per array element

SYNTAX

```
{ $unwind: '$arrayField' }
```

Deconstructs an array — outputs one doc per element. Field name needs a \$ prefix.

```
{ $unwind: '$courseDetails' }
```

BEFORE — array

```
courseDetails: [  
  { course: 'Web Dev' } ]
```

AFTER — flat object

```
courseDetails:  
  { course: 'Web Dev' }
```

⚠️ \$unwind DROPS no-match documents

Students with an empty courseDetails (Neha, Vikram...) vanish. Keep them with: { \$unwind: { path: '\$courseDetails', preserveNullAndEmptyArrays: true } }

Multi-Field Grouping

Group by a combination — make `_id` an object

SYNTAX

```
{ $group: { _id: { f1:'$a', f2:'$b' }, ... } }
```

Make `_id` an OBJECT to group by several fields. One group per unique combination.

```
_id: { city:'$city', subject:'$subject' }
```

```
db.students.aggregate([
  { $group: {
    _id: { city: '$city', subject: '$subject' },    // group by BOTH
    avgMarks: { $avg: '$marks' },
    totalStudents: { $sum: 1 }
  } }
])
// { _id:{ city:'Mumbai', subject:'Math' },    avgMarks:97, totalStudents:1 }
// { _id:{ city:'Delhi',   subject:'Math' },    avgMarks:87.5, totalStudents:2 }
```

Multi-Stage Grouping

Group, then group the result again

```
db.students.aggregate([
  // STAGE 1: group by city + subject
  { $group: { _id: { city: '$city', subject: '$subject' },
    avgPerSubject: { $avg: '$marks' } } },

  // STAGE 2: group the RESULT again, by city only
  { $group: { _id: '$_id.city', // reach into previous _id
    overallAvg: { $avg: '$avgPerSubject' } } },

  { $sort: { overallAvg: -1 } }
])
```

⚠ Keep the \$ in stage 2

`__id: '$_id.city'` ✅ vs `__id: '_id.city'` ❌ (treated as literal text — grouping breaks)

The Full Pipeline — 7 Stages

One real-world reporting query, end to end

1	<code>\$lookup</code>	join students + courses
2	<code>\$unwind</code>	flatten the joined array
3	<code>\$addFields</code>	course, fees + \$cond grade
4	<code>\$group</code>	city stats + \$push students
5	<code>\$sort</code>	cities by avg marks
6	<code>\$project</code>	shape the output
7	<code>\$out</code>	save to finalReport

`$push` collects each group's members into an array — the one new accumulator here.

The Full Aggregation Pipeline

join → flatten → compute → group → sort → format → save (one query, seven stages)

```
db.students.aggregate([

  // 1) JOIN students + courses
  { $lookup: { from:'courses',
    localField:'name',
    foreignField:'studentName',
    as:'courseDetails' } },

  // 2) FLATTEN the joined array
  { $unwind: '$courseDetails' },

  // 3) ADD fields + grade
  { $addFields: {
    course: '$courseDetails.course',
    fees: '$courseDetails.fees',
    grade: {
      $cond: { if:{$gte:['$marks',90]},
        then:'A',
        else:{ $cond:{ if:{$gte:['$marks',80]},
          then:'B',
          else:{ $cond:{ if:{$gte:['$marks',70]},
            then:'C', else:'D' }}}}}
    }
  } },
```

```
// 4) GROUP by city -> stats
{ $group: {
  _id: '$city',
  avgMarks: { $avg: '$marks' },
  maxMarks: { $max: '$marks' },
  minMarks: { $min: '$marks' },
  totalStudents: { $sum: 1 },
  totalFees: { $sum: '$fees' },
  students: { $push: { name:'$name',
    marks:'$marks', grade:'$grade',
    course:'$course' } }
} },

// 5) SORT cities by avg marks
{ $sort: { avgMarks: -1 } },

// 6) FORMAT the output
{ $project: { _id:0, city:'$_id',
  avgMarks:1, maxMarks:1, minMarks:1,
  totalStudents:1, totalFees:1,
  students:1 } },

// 7) SAVE to a new collection
{ $out: 'finalReport' }
])
```

Indexes — What & Why

Like a book's index: jump straight to what you need

An index is a special data structure that lets MongoDB find documents **WITHOUT** scanning the whole collection — like an alphabetical index at the back of a book instead of reading every page.

Without index

Check **EVERY** document one by one. Fine for 15 docs — agonising for 10 million.

With index

Jump straight to the matching documents. Milliseconds, even on huge collections.



MongoDB auto-creates one index on `_id` for every collection. You build the rest.

explain() — Measure the Difference

COLLSCAN vs IXSCAN · totalDocsExamined

SYNTAX

```
db.<col>.find(query).explain('executionStats')
```

Shows how the query ran — stage type, docs returned, and docs EXAMINED.

```
db.employees.find({ department:'Tech' }).explain('executionStats')
```

BEFORE — no index

```
stage: 'COLLSCAN'
      (scans everything)

nReturned: 6

totalDocsExamined: 15
  -> checked ALL 15
      to find 6
```

AFTER — with index

```
db.employees.createIndex(
  { department: 1 })

stage: 'IXSCAN'
nReturned: 6
totalDocsExamined: 6
  -> examined ONLY
      the 6 matches
```

Creating & Managing Indexes

createIndex · getIndexes · dropIndex

SYNTAX

```
createIndex({ field: 1 })
```

Build an index. 1 = ascending, -1 = descending.

```
db.employees.createIndex({ department:1 })
```

SYNTAX

```
getIndexes()
```

List all indexes on the collection.

```
db.employees.getIndexes()
```

SYNTAX

```
dropIndex('name')
```

Remove an index by its name.

```
db.employees.dropIndex('department_1')
```

Index names are auto-generated

```
{ department:1 } -> department_1
```

```
{ city:1, marks:-1 } -> city_1_marks_-1
```



Tradeoff: indexes speed up READS but slow down WRITES and cost storage. Index what you frequently search/sort — not everything.

Compound Index

One index over multiple fields — plus the prefix rule

SYNTAX

```
createIndex({ f1: 1, f2: -1 })
```

Indexes several fields together, in order. Great for combined search + sort.

```
db.students.createIndex({ city:1, marks:-1 })
```

The prefix rule — which queries can use { city:1, marks:-1 }:

```
{ city: 'Delhi' }
```

✓ uses it

city is the leading field

```
{ city, marks }
```

✓ uses it

full match

```
{ marks: 90 }
```

✗ cannot

marks isn't first — prefix broken

Prefix rule: { A, B } helps queries on A and on A+B — but NOT on B alone.

Unique & Text Indexes

Enforce no-duplicates · keyword search

SYNTAX

```
createIndex({ f:1 }, { unique:true })
```

Indexes a field AND rejects duplicate values.

```
createIndex({ name:1 }, { unique:true })
```

SYNTAX

```
createIndex({ f:'text' })
```

Enables keyword search in string fields.

```
createIndex({ name:'text', city:'text' })
```

Unique — duplicate insert is rejected:

```
db.students.insertOne(  
  { name:'Aarav', marks:99 }  
  
// E11000 duplicate key  
// error: name_1  
// -> rejected
```

Text — search with \$text / \$search:

```
db.students.find(  
  $text:{ $search:'Delhi' }  
)  
  
// 'Delhi -Mumbai'  
//   = Delhi but NOT Mumbai
```

💡 One text index per collection (can span fields). Unique index fails to build if duplicates already exist.

Day 5 — In a Nutshell

Joins

```
$lookup  
(join, array)  
$unwind  
(flatten)
```

Grouping

```
multi-field  
_id:{a,b}  
multi-stage  
$_id.field
```

Pipeline

```
lookup->unwind  
->addFields  
->group/$push  
->sort->out
```

Indexes

```
explain()  
COLLSCAN ->  
IXSCAN  
createIndex
```

Index types

```
compound  
(prefix rule)  
unique · text  
($text)
```

Index tradeoff: faster reads, but slower writes + more storage — index the fields you actually query.

Days 1-4

CRUD -> operators -> aggregation

Day 5

Joins, full pipelines, indexes

Next ->

Mongoose · Node/Express · modeling

Ranking & Ties

The SQL RANK() equivalent

Covers:



Why sort+limit cuts ties



Tiebreaker sort (quick fix)



`$setWindowFields`



`$rank · $denseRank · $documentNumber`



Top-N with ties included

`partitionBy` — rank per group

The Problem with limit()

'Top 3' isn't really top 3 when there are ties

limit(3) returns 3 DOCUMENTS — not the top 3 SCORES. If 3rd and 4th are tied, MongoDB keeps one and drops the other, and WHICH one is arbitrary (can change between runs).

Example — Divya and Aryan both scored 93:

```
db.students.find().sort({ marks: -1 }).limit(3)

// Karan(97), Tarun(95), and ... Divya OR Aryan? -> unpredictable
// the other 93 is silently dropped, even though it's tied for 3rd
```



limit(n) = 'give me n rows', NOT 'everyone in the top n scores'. With ties, those differ.

Quick Fix — Tiebreaker Sort

Make the order deterministic

SYNTAX

```
sort({ field1: -1, field2: 1 })
```

Sort by field1; for ties, break by field2. Same result every run.

```
db.students.find().sort({ marks:-1, name:1 }).limit(3)
```

```
db.students.find().sort({ marks: -1, name: 1 }).limit(3)
```

```
// ties on marks now break alphabetically by name -> deterministic
```

Good for a STABLE 3 rows — but it still picks only 3. To INCLUDE everyone tied at the cutoff, you need real ranking (next).

\$setWindowFields — The 3 Rank Functions

Same as SQL window functions; differ only on ties

SYNTAX

```
{ $setWindowFields: { sortBy:{...}, output:{ rnk:{ <fn>:{} } } } }
```

Assigns a rank to each doc based on sortBy. <fn> is one of the three below.

```
output: { rnk: { $denseRank: {} } }
```

MongoDB	SQL	marks 97 95 93 93 91
<code>\$documentNumber</code>	ROW_NUMBER	1 2 3 4 5 (unique)
<code>\$rank</code>	RANK	1 2 3 3 5 (gap after tie)
<code>\$denseRank</code>	DENSE_RANK	1 2 3 3 4 (no gap)

Watch the two tied 93s: docNumber forces 3 & 4 · rank gives 3,3 then skips to 5 · denseRank gives 3,3 then 4.

Top 3 Scores — Ties Included

\$denseRank + \$match — the real 'top N'

```
db.students.aggregate([
  { $setWindowFields: {
    sortBy: { marks: -1 },
    output: { rnk: { $denseRank: {} } }
  } },
  { $match: { rnk: { $lte: 3 } } }
])
```

Returns everyone in the top 3 distinct scores. If two students tie at 93 (3rd), BOTH appear — you might get 4 docs for 'top 3', exactly like SQL DENSE_RANK.

Exactly 3 rows

\$documentNumber

Top 3, gaps after ties

\$rank

Top 3 distinct, no gaps

\$denseRank

Ranking Within Groups

partitionBy — like SQL's PARTITION BY

SYNTAX

```
{ $setWindowFields: { partitionBy: '$field', sortBy:{...}, output:{...} } }
```

Ranking RESTARTS for each partition (group). Each city gets its own 1, 2, 3.

```
partitionBy: '$city'
```

```
// top scorer in EACH city
db.students.aggregate([
  { $setWindowFields: {
    partitionBy: '$city',           // restart rank per city
    sortBy: { marks: -1 },
    output: { rnk: { $denseRank: {} } } },
  { $match: { rnk: 1 } }           // rank 1 in each city
])
```

📌 Keep the \$ — partitionBy: '\$city', not 'city'. Without partitionBy, ranking runs across the whole collection.

Ranking — In a Nutshell

The trap

```
sort+limit  
= n rows,  
not n scores  
ties cut
```

Quick fix

```
tiebreaker  
sort field  
-> stable,  
still n rows
```

Real ranking

```
$setWindow  
Fields  
+ $match  
rnk <= 3
```

3 functions

```
docNumber  
=ROW_NUMBER  
$rank=RANK  
dense=DENSE
```

Per group

```
partitionBy  
'$city'  
restarts  
ranking
```

Bottom line: `limit()` is fine for 'some n rows'. For true ranking with ties, use `$setWindowFields` + `$denseRank`. Requires MongoDB 5.0+.